# CASO #1 - EJERCICIO #2

**Author:** José Gabriel Marín Aguilar c.2022119819

Viernes 6 de marzo de 2026

## Problem Overview

The Documento Único Aduanero (DUA) is the official customs declaration document required for import and export operations in Costa Rica, regulated by the Ministerio de Hacienda.

Preparing a DUA requires interpreting heterogeneous documents such as invoices, packing lists, certificates of origin, transport documents, and permits. These files vary in format (Excel, Word, PDF, scanned images), structure, terminology, and quality. Manual preparation is:

- Repetitive
- Error-prone
- Highly dependent on expert knowledge
- Time-consuming

## Proposed Solution

DUA Streamliner is an AI-powered system that ingests a folder containing heterogeneous trade documents, extracts relevant customs data using semantic models, maps the information into the official DUA template, and generates a pre-filled Word document with confidence indicators.

The system does not replace the customs expert; it reduces operational workload and shifts the expert's role toward validation and decision-making.

| Pros | Cons |
|---|---|
| Reduces manual workload | OCR accuracy depends on document quality |
| Minimizes data entry errors | AI extraction requires continuous validation |
| Standardizes DUA preparation | Legal responsibility remains with the declarant |
| Improves processing time | |

# 1. Frontend Design

## 1.1 Technology Framework

- **Application type:** Web application fully responsive for tablet and mobile access.
- **Web Framework:** React 19.2
- **Runtime:** Node.js 21.x
- **Coding Language:** TypeScript 5.9.3
- **Bundler:** Vite 6.x

- **UI Framework:** MUI 6.x
- **Routing:** React Router DOM 6.25.x
- **Authentication Protocol:** OAuth2 / OpenID Connect (OIDC Client TS 3.x)
- **Unit Testing:** Jest 30.2.0
- **Data Validation:** Zod 3.x
- **Code prettier framework:** Prettier 3.x
- **Code style framework:** ESLint 9.x with @typescript-eslint 7.x
- **Integration testing tools:** Playwright 1.58.2
- **Cloud service:** Amazon Web Services (AWS)
- **Hosted services within the cloud service:**
  - AWS ECS Fargate
  - AWS Elastic Container Registry (ECR)
  - AWS Application Load Balancer (ALB)
  - AWS CloudFront
  - AWS CloudWatch
  - AWS Certificate Manager (ACM)
- **Code repositories service:** GitHub
- **Code automation task tool:** npm scripts (Node.js 21.x runtime)
- **CI CD pipelines technology:** GitHub Actions
- **Environments:**
  - Development
  - Quality Assurance
  - Stage
  - Production
- **Environment deployment tools:**
  - Docker 25.x
  - Terraform 1.8.x
  - AWS CLI 2.x
- **Observability framework:**
  - AWS CloudWatch
  - OpenTelemetry JS 1.x
  - Sentry

---

## 1.2 UX/UI Analysis

### Core Business Process

The application allows customs operators to automatically generate a Documento Único Aduanero (DUA) from multiple source documents.

Users authenticate using their credentials and a one-time token sent to their email. Once logged in, they can either review previously generated DUAs or initiate the creation of a new one.

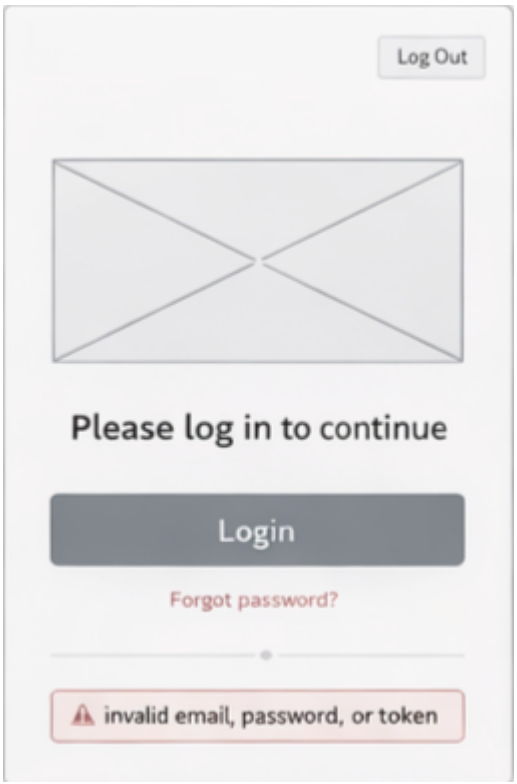
When generating a new DUA, the user configures the generation parameters such as accepted document formats and the template version to be used. The system then processes the uploaded documents, extracts the relevant customs information using AI models, and maps the data into the official DUA structure.

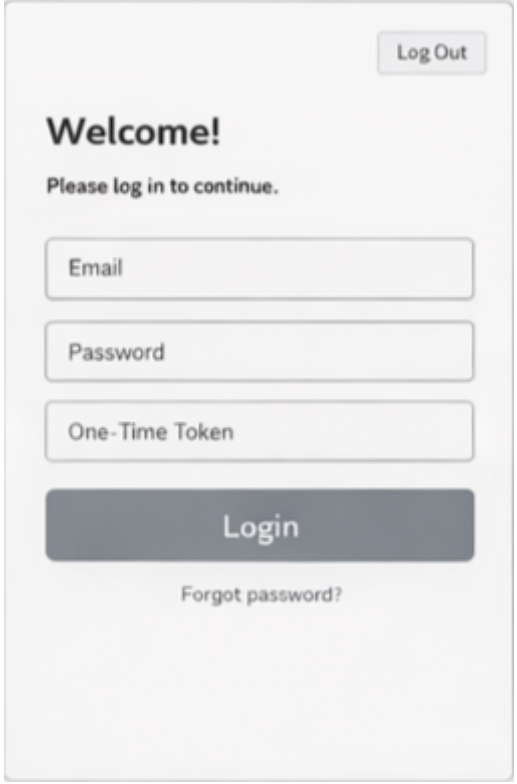
During processing, the user can monitor the progress of the extraction and generation process. Once completed, the system displays the generated DUA in a PDF viewer where the user can review and download the final document.

## Wireframes

**User Login**

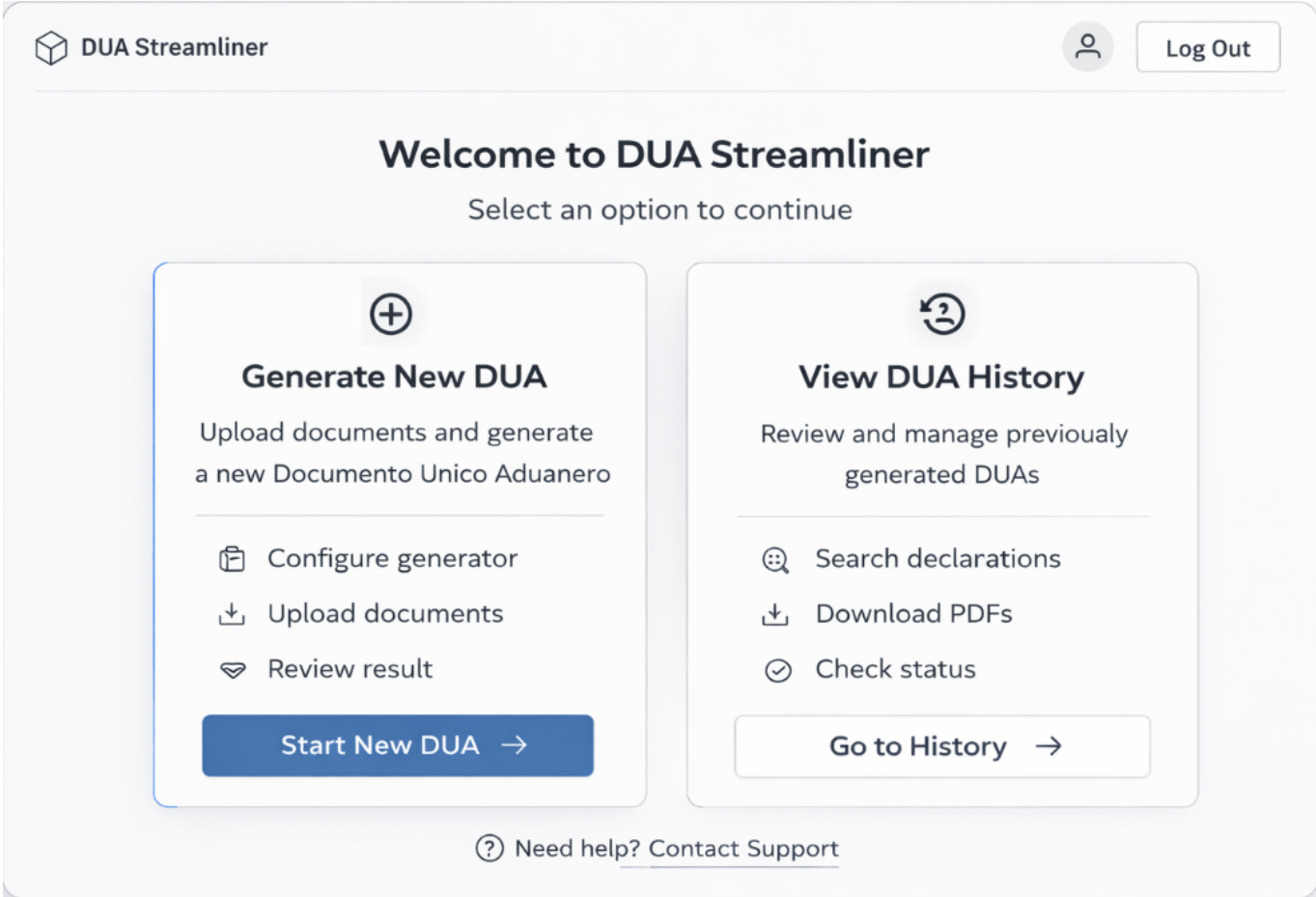
The user logs into the system using their username, password, and a one-time authentication token received via email.



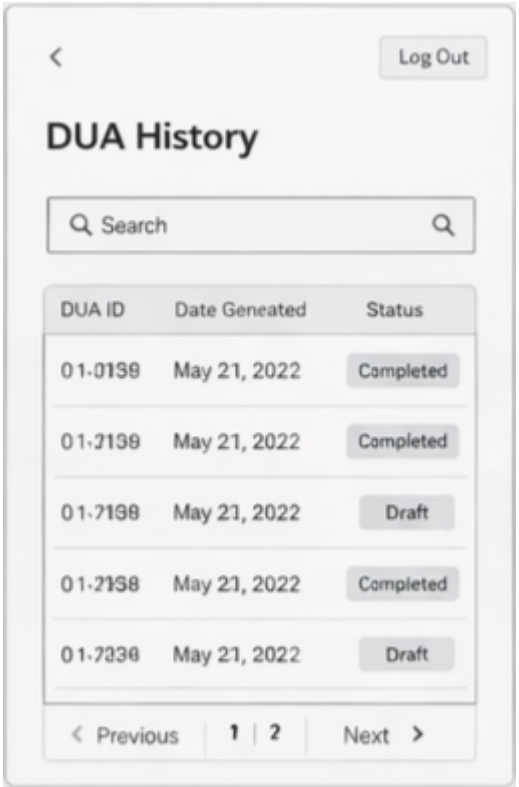

**Select Option**

After authentication, the user can choose between generating a new DUA or reviewing previously generated declarations.




**DUA History**

The system displays a list of previously generated DUAs associated with the user account. Each record can later be expanded or downloaded.

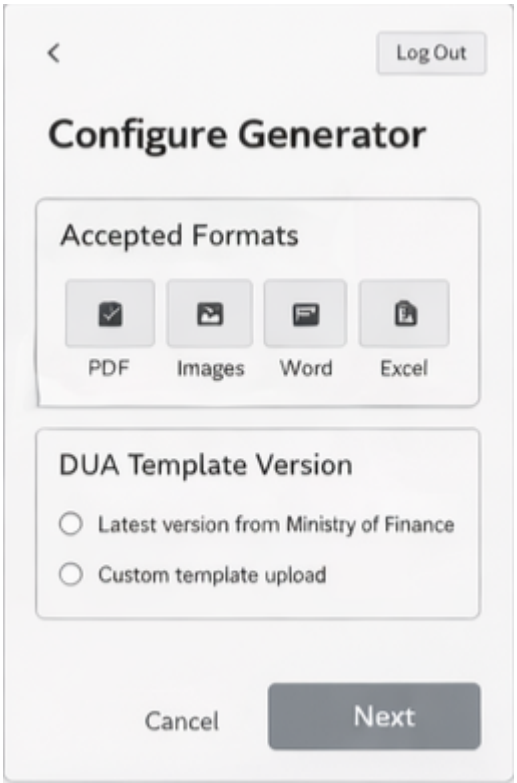



**Generator Configuration**

The user configures the generation process before uploading documents.

Configuration options include:

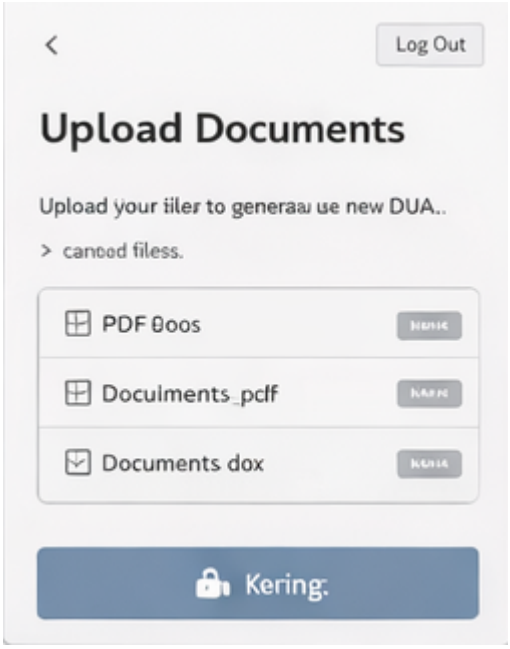
- Accepted document formats (PDF, Image, Word, Excel)
- DUA template version (latest from the Ministry of Finance or custom uploaded template)
- AI extraction mode (fast scan / detailed extraction)

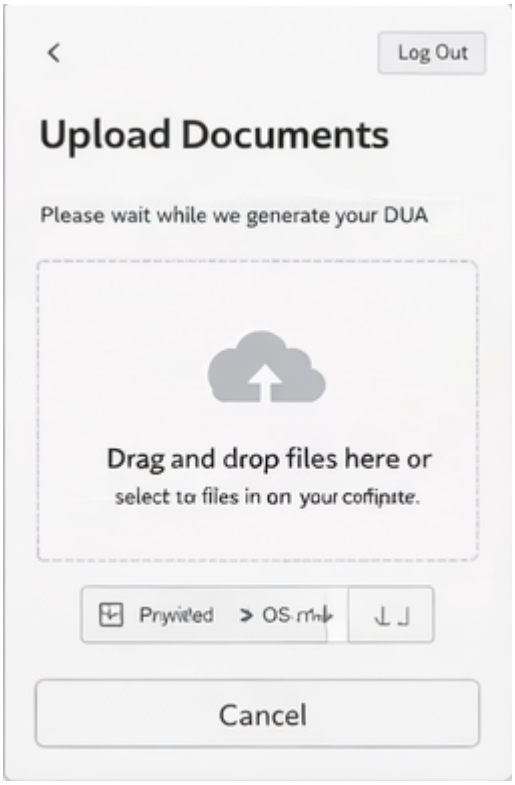


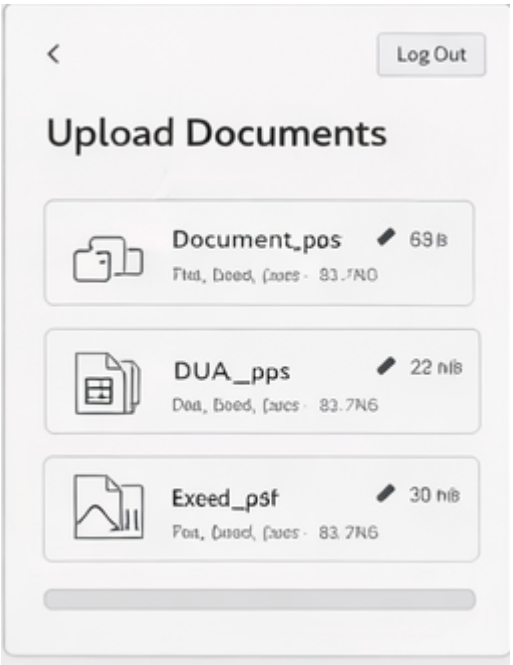

**Upload Files**

The user uploads the commercial documents required to generate the DUA. Files can be uploaded using drag-and-drop or manual selection. Supported formats include:

- PDF
- DOCX
- XLSX
- Images

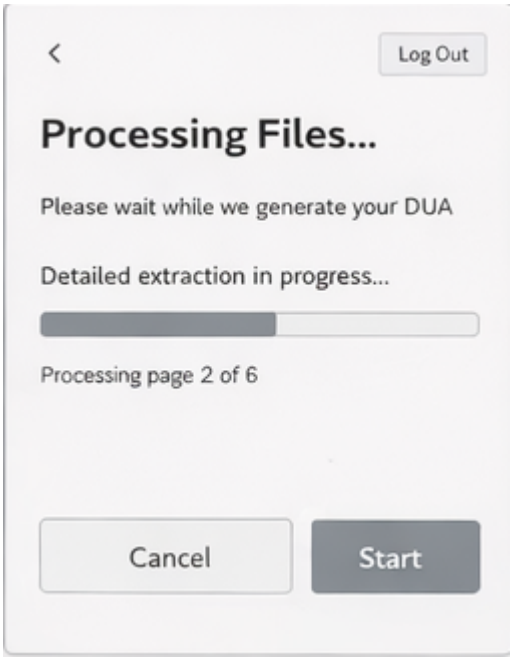



**Processing Progress**

During the document processing stage, the user can monitor the progress of the generation pipeline. The interface displays a progress bar and indicates the current step of the process.




**DUA Result**

Once the process is completed, the generated DUA is displayed in a PDF preview interface. The user can review the document and download it as a PDF file.

## Modelo alternativo

Un modelo alternativo para ver los wireframes puede encontrarse en la siguiente página: Wireframes generados a partir de Coderick AI

---

## 1.3

---

## 1.4 Security

Security in the DUA Streamliner platform is designed following modern web application security principles, focusing on secure authentication, controlled authorization, and safe management of sensitive information. The system combines token-based authentication, role-based authorization, and secure storage of credentials to protect both user data and system operations.

Authentication

User authentication is implemented using a token-based mechanism with JSON Web Tokens (JWT). When a user logs in using their credentials (email and password), the backend validates the credentials and generates a signed JWT token that represents the authenticated session. This token is then sent to the client and included in subsequent API requests to verify the identity of the user.

Passwords are never stored in plain text. Instead, they are securely hashed using industry-standard algorithms (such as bcrypt) before being stored in the database. This ensures that even if the database were compromised, user credentials would remain protected.

Although the current implementation uses a single-factor authentication (password-based login), the architecture allows the future integration of Multi-Factor Authentication (MFA). MFA would add an additional verification step such as a One-Time Password (OTP) sent via email or generated by an authenticator application, significantly increasing account security.

Single Sign-On Considerations

The system currently uses an independent authentication mechanism rather than a Single Sign-On (SSO) provider. This approach simplifies the architecture and allows full control over user management inside the application.

However, the architecture is compatible with future integration of third-party identity providers such as Google authentication. Implementing SSO could improve user convenience and reduce password management risks, but for the scope of this project an internal authentication system provides sufficient control and security.

Session Management

Session management is handled using stateless authentication tokens. After successful login, the backend issues a JWT token containing the user's identity and authorization claims. This token has a defined expiration time and must be included in the header of protected API requests.

Using stateless tokens improves scalability and simplifies backend session management since the server does not need to maintain session state. Token expiration and optional refresh tokens help prevent long-term misuse if a token is compromised.

Authorization

Authorization in the system is implemented using Role-Based Access Control (RBAC). Each authenticated user is assigned a role, and each role defines the actions that the user is allowed to perform within the platform.

Typical roles within the system include:

Importer / Exporter – Users who generate and manage DUA declarations.

Customs Officer – Users who review submitted declarations.

Administrator – Users who manage system configurations and permissions.

RBAC simplifies permission management while maintaining clear separation of responsibilities between different user groups. The authorization rules are enforced in the backend through middleware or guards that validate the user's role and permissions before executing protected operations.

Access Control Model

The platform primarily uses RBAC for authorization. Access to the system itself is controlled through authenticated accounts rather than open public access lists. Therefore, only registered users with valid credentials can access the platform.

Although RBAC is sufficient for the current scope, the architecture allows extending the model to include Policy-Based Access Control (PBAC) if more complex access rules become necessary in the future.

Permissions and Claims

Permissions within the system are defined using structured claims, which represent the actions a user can perform. These claims can be embedded inside the authentication token or validated on the server side.

| Code | Description |
|---|---|
| CREATE-DUA | Allows the user to generate a new DUA declaration. |
| UPLOAD-DOCUMENT | Allows the user to upload supporting documents for a declaration. |
| VIEW-DUA | Allows the user to view previously generated declarations. |
| EDIT-DUA | Allows the user to modify declaration data before submission. |
| REVIEW-DUA | Allows customs officers to review submitted declarations. |
| ADMIN-MANAGE-USERS | Allows administrators to manage user accounts and roles. |

These claims are evaluated by the backend authorization layer to ensure that users only perform actions allowed by their role.

Secure Storage of Sensitive Data

Sensitive configuration data such as API keys, database credentials, authentication secrets, and environment variables are not stored directly in the source code repository. Instead, they are managed using environment variables stored in a .env configuration file.

For deployment environments, sensitive values should be stored using a secure secrets management service, such as a cloud-based key vault. These services securely store encryption keys, tokens, and credentials while restricting access through controlled permissions.

Typical sensitive data managed in secure storage include:

JWT signing keys

Database connection credentials

API keys for external services

Environment-specific configuration variables

This approach prevents exposure of sensitive data in version control systems and ensures that secrets can be rotated or revoked securely.